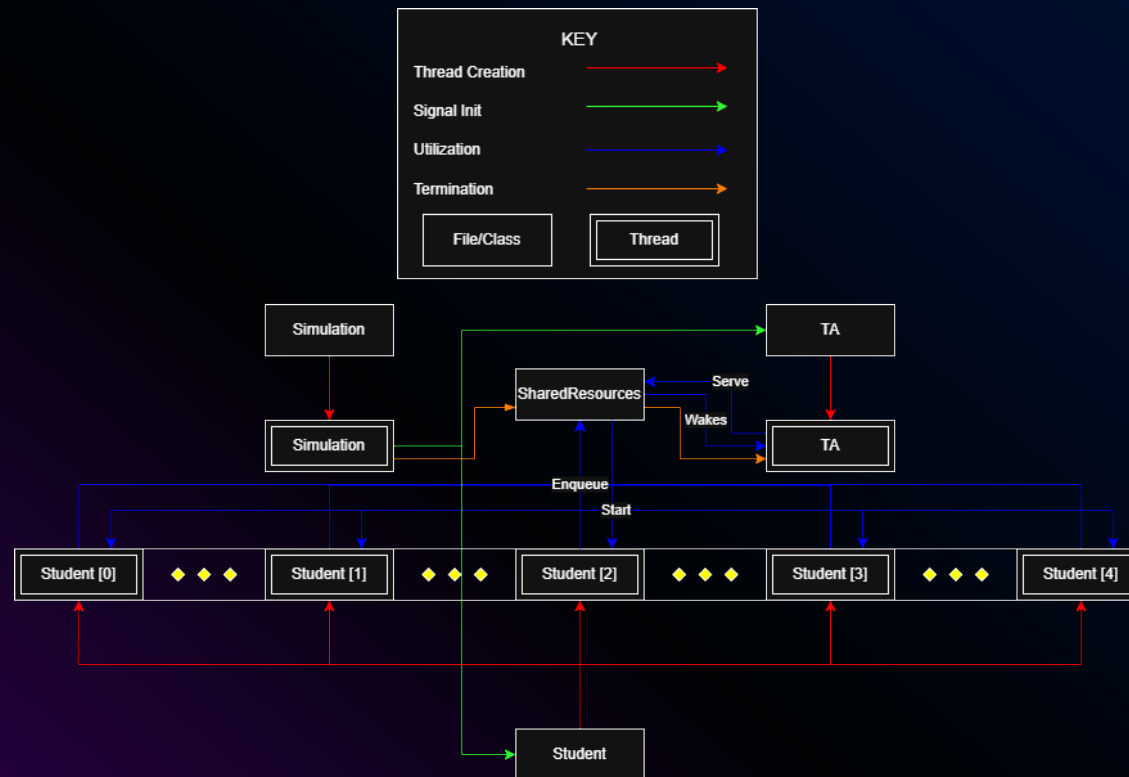


CRITICAL SECTION AND MUTUAL EXCLUSION

DEMONSTRATING SEMAPHORES AND MUTEX
LOCKS IN C++

SYSTEM MAP



SYSTEM MAP (CONTINUED)

- Simulation class creates the Simulation thread, initializes the SharedResources class and calling on the TA and Student classes to make their respective threads.
- All thread interactions pass through SharedResources, including enqueue and dequeue, waking TA, and signaling
- Simulation sets the requestShutdown flag, which passes through SharedResources, terminating the TA thread. Afterwards, Simulation conducts join, and terminates the program

SIMULATION (INITIALIZATION)

```
void Simulation::setup() {  
    // Build shared state from the config (chairs, students, etc.)  
    shared_ = std::make_unique<SharedResources>(*config_);  
  
    // Optional logging cosmetics  
    Logger::instance().setTimestamps(true);  
    Logger::instance().setThreadIds(true);  
}  
  
void Simulation::startTA() {  
    ta_ = std::make_unique<TA>(*shared_, *config_);  
    taThread_ = std::thread(&TA::serviceLoop, ta_.get());  
}  
  
void Simulation::startStudents() {  
    const int n = static_cast<int>(config_>numStudents());  
    students_.reserve(n);  
    studentThreads_.reserve(n);  
  
    for (int i = 0; i < n; ++i) {  
        students_.push_back(std::make_unique<Student>(i, *shared_, *config_));  
        students_.back()->setName("Student-" + std::to_string(i));  
        studentThreads_.emplace_back(&Student::run, students_.back().get());  
    }  
}
```

- Fairly simple code section – builds the environment for everything else to run in.
- setup() starts the config_ (parameters/timing) and the shared_ (queue and semaphores)
- startTA() triggers the TA class to create the TA worker thread
- startStudents() similarly triggers the Student class to create the Student worker threads

TA

```
void TA::serviceLoop() {
    try {
        for (;;) {
            onIdleStart();
            if (!waitForWorkOrShutdown()) break; // blocks for a permit (or exits)
            onIdleEnd();

            // Serve the student for the permit we just acquired
            {
                int id = dequeueNextStudent();
                helpOneStudent(id);
            }

            // Drain any queued permits that arrived mid-help
            while (shared_.tryAcquireWorkPermit()) {
                int id = dequeueNextStudent();
                if (id < 0) break; // sanity check
                helpOneStudent(id);
            }

            Logger::instance().infoC(name_, "exiting; total helps=" + std::to_string(helpsServed_));
        }
    } catch (const std::exception& ex) {
        Logger::instance().infoC(name_, std::string("FATAL: unhandled exception: ") + ex.what());
    } catch (...) {
        Logger::instance().infoC(name_, "FATAL: unhandled non-std exception");
    }
}
```

- dequeueNextStudent() and helpOneStudent(id) iterates the student queue and helps each student as required
- shared_.tryAcquireWorkPermit() attempts to non-blockingly acquire a work permit from the counting semaphore; if none are available, it indicates that the queue is empty
- onIdleStart() and onIdleEnd() logs and tracks TA idle (nap) time
- waitForWorkOrShutdown() checks shutdown() flag and queue to wake TA – if shutdown() flag is set, finishes queue then terminates TA thread

STUDENT

```
void Student::run() {
    try {
        const auto target = cfg_.maxHelpsPerStudent();
        while (helpsReceived_ < target) {
            thinkOnce();
            if (!tryEnqueue()) {
                std::this_thread::sleep_for(cfg_.randRetryBackoff());
                continue;
            }
            waitForHelpAndAcknowledge();
            ++helpsReceived_;
        }
    }
    catch (const std::exception& ex) {
        Logger::instance().infoC(name_, std::string("FATAL: unhandled exception: ") + ex.what());
    }
    catch (...) {
        Logger::instance().infoC(name_, "FATAL: unhandled non-std exception");
    }
}
```

- thinkOnce() simulates the student working on their own
- tryEnqueue() tries to place the student into the queue, returning false if the queue is full, which triggers the randRetryBackoff(), which returns the “sleep” duration for the student before re-attempting to enter the queue
- waitForHelpAndAcknowledge() blocks the student until the TA calls signalStart(), then immediately signals signalDone() to acknowledge completion.

CRITICAL SECTION AND MUTUAL EXCLUSION

There are two critical sections in `SharedResources`: one in `enqueueIfSpace()` and one in `dequeueStudent()`. Both are protected by the same mutex `mtx_`, ensuring mutual exclusion whenever the queue is accessed. In each function, `std::scoped_lock lk(mtx_);` acquires the mutex (blocking until available) and automatically releases it at scope exit. In `enqueueIfSpace()`, the queue push and the `workPermits_.release()` occur inside the same critical section, so every permit corresponds to exactly one queued student. In `dequeueStudent()`, the TA safely pops one student while holding the mutex, preventing races with concurrent enqueues.

```
bool SharedResources::enqueueIfSpace(int studentId) {  
    std::scoped_lock lk(mtx_);  
    if (queue_.size() == capacity_) return false;  
    queue_.push(studentId);  
    trackQueueHighWater_();  
    workPermits_.release(); // wake TA (or buffer a permit)  
    return true;  
}
```

These critical sections ensure that no two threads can modify the queue simultaneously, preventing race conditions and maintaining a consistent one-to-one mapping between enqueued students and work permits.

```
int SharedResources::dequeueStudent() {  
    std::scoped_lock lk(mtx_);  
    if (queue_.empty())  
        return -1;  
    int id = queue_.front();  
    queue_.pop();  
    return id;  
}
```

COUNTING SEMAPHORE USE

workPermits_ is a counting semaphore that synchronizes Student and TA activity. Students release a permit when they enqueue, and the TA acquires a permit before serving - ensuring perfect one-to-one coordination between waiting students and available work.

Implemented as `std::counting_semaphore<INT_MAX>` workPermits_, initialized to zero.

```
void SharedResources::acquireWorkPermit() {  
    workPermits_.acquire();  
}  
  
bool SharedResources::tryAcquireWorkPermit() {  
    return workPermits_.try_acquire();  
}  
  
bool SharedResources::hasWorkPermit() const {  
    // Non-destructive peek  
    auto* sem = const_cast<std::counting_semaphore<INT_MAX>*>(&workPermits_);  
    if (sem->try_acquire()) { sem->release(); return true; }  
    return false;  
}
```


BINARY SEMAPHORES

StudentSync uses two binary semaphores, helpStart_ and helpDone_, to provide a syncing mechanism between the Student threads and the TA thread.

Each StudentSync object is dedicated to one student, ensuring TA↔Student coordination without shared contention.

```
// TA -> Student: invite
void StudentSync::signalStart() { helpStart_.release(); }
// TA <- Student: done
void StudentSync::waitDone() { helpDone_.acquire(); }

// Student waits until TA invites
void StudentSync::waitForStart() { helpStart_.acquire(); }
// Student tells TA they're finished
void StudentSync::signalDone() { helpDone_.release(); }
```

SIMULATION (TERMINATION)

When all Student threads complete their work, the Simulation thread cleanly terminates the TA thread and finalizes metrics.

- `joinStudents()` waits for all student threads to finish work
- `shutdownTA()` sets the shutdown flag and wakes the TA if sleeping, allowing it to exit cleanly.
- The Simulation thread then joins the TA and Student threads, prints a summary of metrics, and confirms orderly synchronization and full shutdown, exiting the simulation

```
void Simulation::joinStudents() {  
    for (auto& t : studentThreads_) {  
        if (t.joinable()) t.join();  
    }  
}  
  
void Simulation::shutdownTA() {  
    // Tell TA to exit once the queue drains; also wake it if blocked  
    shared_>requestShutdown();  
    shared_>nudgeTA();  
    if (taThread_.joinable()) taThread_.join();  
}
```

OUTPUT

```
[17:51:32.549] [TID 808463C09C7EA36] [TA] no students; taking a nap
[17:51:32.776] [TID DE15ED24EDA02CE1] [Student-0] sat in hallway (enqueued)
[17:51:32.777] [TID DE15ED24EDA02CE1] [Student-0] waiting for TA
[17:51:32.778] [TID 808463C09C7EA36] [TA] woken up
[17:51:32.778] [TID 808463C09C7EA36] [TA] inviting Student 0
[17:51:32.779] [TID DE15ED24EDA02CE1] [Student-0] finished; leaving hallway
[17:51:33.111] [TID B864ADD789833362] [Student-2] sat in hallway (enqueued)
[17:51:33.112] [TID B864ADD789833362] [Student-2] waiting for TA
[17:51:33.159] [TID D131669061E32A9] [Student-1] sat in hallway (enqueued)
[17:51:33.160] [TID D131669061E32A9] [Student-1] waiting for TA
[17:51:33.174] [TID 808463C09C7EA36] [TA] inviting Student 2
[17:51:33.176] [TID B864ADD789833362] [Student-2] finished; leaving hallway
[17:51:33.190] [TID 73D6743CD7D7B244] [Student-5] sat in hallway (enqueued)
[17:51:33.191] [TID 73D6743CD7D7B244] [Student-5] waiting for TA
[17:51:33.204] [TID 37300F3D70980B76] [Student-3] sat in hallway (enqueued)
[17:51:33.205] [TID 37300F3D70980B76] [Student-3] waiting for TA
[17:51:33.505] [TID 808463C09C7EA36] [TA] inviting Student 1
[17:51:33.506] [TID D131669061E32A9] [Student-1] finished; leaving hallway
[17:51:33.681] [TID B864ADD789833362] [Student-2] sat in hallway (enqueued)
[17:51:33.682] [TID B864ADD789833362] [Student-2] waiting for TA
[17:51:33.905] [TID 808463C09C7EA36] [TA] inviting Student 5
[17:51:33.906] [TID 73D6743CD7D7B244] [Student-5] finished; leaving hallway
[17:51:33.921] [TID 6E725C95F9F582C0] [Student-4] sat in hallway (enqueued)
[17:51:33.922] [TID 6E725C95F9F582C0] [Student-4] waiting for TA
[17:51:34.790] [TID 808463C09C7EA36] [TA] inviting Student 3
[17:51:34.791] [TID 37300F3D70980B76] [Student-3] finished; leaving hallway
```

```
[17:51:34.886] [TID DE15ED24EDA02CE1] [Student-0] sat in hallway (enqueued)
[17:51:34.888] [TID DE15ED24EDA02CE1] [Student-0] waiting for TA
[17:51:35.404] [TID 808463C09C7EA36] [TA] inviting Student 2
[17:51:35.406] [TID B864ADD789833362] [Student-2] finished; leaving hallway
[17:51:35.544] [TID 73D6743CD7D7B244] [Student-5] sat in hallway (enqueued)
[17:51:35.545] [TID 73D6743CD7D7B244] [Student-5] waiting for TA
[17:51:36.145] [TID 808463C09C7EA36] [TA] inviting Student 4
[17:51:36.146] [TID 6E725C95F9F582C0] [Student-4] finished; leaving hallway
[17:51:36.239] [TID D131669061E32A9] [Student-1] sat in hallway (enqueued)
[17:51:36.240] [TID D131669061E32A9] [Student-1] waiting for TA
[17:51:37.050] [TID 808463C09C7EA36] [TA] inviting Student 0
[17:51:37.051] [TID DE15ED24EDA02CE1] [Student-0] finished; leaving hallway
[17:51:37.368] [TID 6E725C95F9F582C0] [Student-4] sat in hallway (enqueued)
[17:51:37.369] [TID 6E725C95F9F582C0] [Student-4] waiting for TA
[17:51:37.762] [TID 808463C09C7EA36] [TA] inviting Student 5
[17:51:37.763] [TID 73D6743CD7D7B244] [Student-5] finished; leaving hallway
[17:51:38.205] [TID 37300F3D70980B76] [Student-3] sat in hallway (enqueued)
[17:51:38.206] [TID 37300F3D70980B76] [Student-3] waiting for TA
[17:51:38.504] [TID 808463C09C7EA36] [TA] inviting Student 1
[17:51:38.505] [TID D131669061E32A9] [Student-1] finished; leaving hallway
[17:51:39.263] [TID 808463C09C7EA36] [TA] inviting Student 4
[17:51:39.265] [TID 6E725C95F9F582C0] [Student-4] finished; leaving hallway
[17:51:39.771] [TID 808463C09C7EA36] [TA] inviting Student 3
[17:51:39.772] [TID 37300F3D70980B76] [Student-3] finished; leaving hallway
[17:51:40.632] [TID 808463C09C7EA36] [TA] no students; taking a nap
[17:51:40.634] [TID 808463C09C7EA36] [TA] exiting; total helps=12
```


SIMULATION SUMMARY AND RESULTS

```
[17:51:40.634] [TID 2361C52514E02239] =====
[17:51:40.635] [TID 2361C52514E02239]           Simulation Summary
[17:51:40.635] [TID 2361C52514E02239] =====
[17:51:40.636] [TID 2361C52514E02239] Students           : 6
[17:51:40.636] [TID 2361C52514E02239] Chairs (capacity)  : 3
[17:51:40.637] [TID 2361C52514E02239] Total helps served : 12
[17:51:40.637] [TID 2361C52514E02239] Max queue depth    : 3
[17:51:40.638] [TID 2361C52514E02239] Queue at shutdown  : 0
[17:51:40.638] [TID 2361C52514E02239] TA idle time (ms)   : 228
[17:51:40.638] [TID 2361C52514E02239] Wall time (ms)      : 8087
[17:51:40.639] [TID 2361C52514E02239] TA idle percent     : 2.819340%
```

For each line, you can see the timestamp of the logged output, followed by the active thread. For the worker (Student and TA) threads, you next see the active thread's label (TA, or Student-<ID>), followed by the action taken. For the simulation thread, it covers the summary of the simulation.

This log sample is with standard verbosity, if run on command line, it can be run with ``-v2`` to increase verbosity, also logging events such as a student attempting to enter a full queue or when the student is actively working with the TA.

The final summary confirms correct synchronization - all students were served, queue never exceeded capacity, and TA idle time remained low (~4%), demonstrating efficient thread coordination.